Exercício Prático Guiado — Sistema de Gestão de Tarefas com EDA, Spring Boot e RabbitMQ (Docker Local)

Guia de Estudo para o Trabalho Prático

1. Visão Geral do Exercício

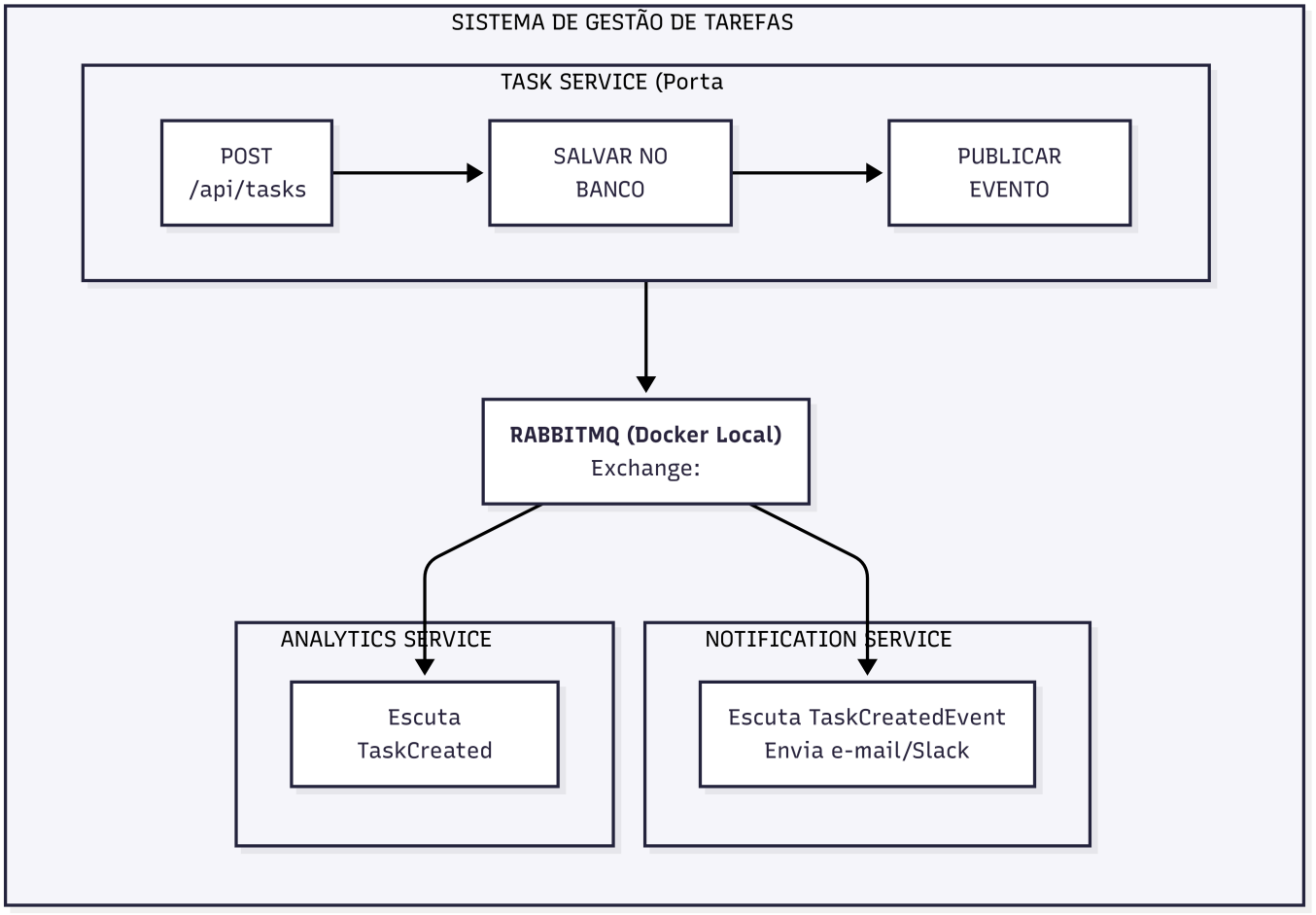
Item	Descrição
Objetivo	Construir um sistema de gestão de tarefas utilizando Arquitetura Orientada a Eventos com Spring Boot e RabbitMQ em ambiente Docker local, servindo como base para o trabalho prático em grupo.
Tecnologias	Java 21, Spring Boot 4.x, Spring AMQP, Spring Data JPA, H2 Database, Docker Desktop com WSL2
Duração estimada	90–120 minutos
Pré-requisitos	Docker Desktop instalado e em execução, JDK 21, IDE (IntelliJ/Eclipse/VS Code)

Cenário do Sistema

O sistema de gestão de tarefas permite que usuários criem tarefas. Cada tarefa criada dispara eventos que são processados por diferentes serviços:

1. **Task Service** → Cria a tarefa e publica um evento `TASK_CREATED`.
2. **Notification Service** → Escuta o evento e envia notificações (simuladas) sobre a nova tarefa.
3. **Analytics Service** → Escuta o evento e registra métricas (contagem de tarefas por prioridade).

Arquitetura do Exercício:



---

2. Configuração do Ambiente com Docker

Passo 2.1 — Criar o arquivo docker-compose.yml

Crie uma pasta para o projeto (ex: `task-management-system`) e dentro dela crie o arquivo `docker-compose.yml`:

```
version: '3.8'

services:
  rabbitmq:
    image: rabbitmq:3-management-alpine
    container_name: task-rabbitmq
    ports:
      - "5672:5672"      # Porta para conexão do Spring Boot
      - "15672:15672"    # Porta para a interface web (Management UI)
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin
    volumes:
      - rabbitmq_data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmqctl", "status"]
      interval: 10s
      timeout: 5s
```

```
      retries: 5

volumes:
  rabbitmq_data:
```

Passo 2.2 — Iniciar o RabbitMQ

Passo	Comando	Descrição
1	Abra o terminal (PowerShell ou WSL)	Navegue até a pasta com o `docker-compose.yml`
2	`docker-compose up -d`	Inicia o RabbitMQ em segundo plano
3	`docker-compose ps`	Verifica se o container está rodando
4	`docker-compose logs rabbitmq`	(Opcional) Verifica os logs do RabbitMQ

Passo 2.3 — Acessar a Interface de Gerenciamento do RabbitMQ

Passo	Ação
1	Abra o navegador e acesse: `http://localhost:15672`
2	Login: `admin`
3	Explore as abas: **Connections**, **Exchanges**, **Queues** (todas vazias por enquanto)

---

3. Estrutura do Projeto

Crie **3 projetos Spring Boot** separados (ou módulos dentro de um projeto Maven/Gradle):

```
task-management-system/
├── task-service/           (Porta 8081)
├── notification-service/   (Porta 8082)
├── analytics-service/      (Porta 8083)
└── docker-compose.yml
```

3.1. Criar os projetos no Spring Initializr

Passo	Ação
1	Acesse start.spring.io
2	Para cada serviço, selecione: **Java 21**, **Spring Boot 3.x**, **Maven**
3	Adicione as dependências: **Spring Web**, **Spring for RabbitMQ**, **Spring Data JPA**, **H2 Database**, **Lombok**
4	Gere e baixe cada projeto
5	Extraia todos na pasta `task-management-system/`

3.2. Organização dos Eventos (DTOs compartilhados)

Copie as classes de evento em cada serviço (em projetos reais, usa-se um módulo comum, mas para este exercício faremos a cópia).

Estrutura de pastas (exemplo para task-service):

```
task-service/
├── src/main/java/com/example/task/
│   ├── TaskServiceApplication.java
│   ├── config/RabbitMQConfig.java
│   ├── controller/TaskController.java
│   ├── model/Task.java
│   ├── model/TaskPriority.java
│   ├── model/TaskStatus.java
│   ├── repository/TaskRepository.java
│   ├── service/TaskService.java
│   └── event/
│       ├── TaskEvent.java
│       └── TaskCreatedEvent.java
└── src/main/resources/application.properties
```

---

4. Implementação do Task Service (Produtor)

4.1. application.properties

```
spring.application.name=task-service
server.port=8081

# RabbitMQ (Docker Local)
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=admin
spring.rabbitmq.password=admin

# H2 Database (arquivo)
spring.datasource.url=jdbc:h2:file:./data/taskdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXI
T=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# RabbitMQ – Exchange e Routing Keys
```

```
rabbitmq.exchange.name=task.exchange
rabbitmq.routing.created=task.created
```

4.2. Modelos (Task.java, TaskPriority.java, TaskStatus.java)

```
// Task.java
package com.example.task.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;

@Entity
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String description;
    private String assignee;
    @Enumerated(EnumType.STRING)
    private TaskPriority priority;
    @Enumerated(EnumType.STRING)
    private TaskStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}

// TaskPriority.java
package com.example.task.model;
public enum TaskPriority { LOW, MEDIUM, HIGH, CRITICAL }

// TaskStatus.java
package com.example.task.model;
public enum TaskStatus { PENDING, IN_PROGRESS, DONE, CANCELLED }
```

4.3. Repositório (TaskRepository.java)

```
package com.example.task.repository;

import com.example.task.model.Task;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface TaskRepository extends JpaRepository<Task, Long> {
}
```

4.4. Eventos (TaskEvent.java e TaskCreatedEvent.java)

```
// TaskEvent.java (Classe Base)
package com.example.task.event;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;
import java.util.UUID;

@Data
@AllArgsConstructor
@NoArgsConstructor
public abstract class TaskEvent {
    private String eventId;
    private String eventType;
    private Long taskId;
    private LocalDateTime timestamp;

    public TaskEvent(String eventType, Long taskId) {
        this.eventId = UUID.randomUUID().toString();
        this.eventType = eventType;
        this.taskId = taskId;
        this.timestamp = LocalDateTime.now();
    }
}

// TaskCreatedEvent.java
package com.example.task.event;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
@EqualsAndHashCode(callSuper = true)
public class TaskCreatedEvent extends TaskEvent {
    private String title;
    private String description;
    private String assignee;
    private String priority;

    public TaskCreatedEvent(Long taskId, String title, String description,
                            String assignee, String priority) {
```

```
        super("TASK_CREATED", taskId);
        this.title = title;
        this.description = description;
        this.assignee = assignee;
        this.priority = priority;
    }
}
```

4.5. Configuração do RabbitMQ (RabbitMQConfig.java)

```
package com.example.task.config;

import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    @Value("${rabbitmq.exchange.name}")
    private String exchangeName;

    @Bean
    public TopicExchange taskExchange() {
        return new TopicExchange(exchangeName);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory) {
        RabbitTemplate template = new RabbitTemplate(connectionFactory);
        template.setMessageConverter(jsonMessageConverter());
        return template;
    }
}
```

4.6. Serviço de Tarefas (TaskService.java)

```
package com.example.task.service;
```

```
import com.example.task.event.TaskCreatedEvent;
import com.example.task.model.Task;
import com.example.task.model.TaskStatus;
import com.example.task.repository.TaskRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

@Service
@RequiredArgsConstructor
@Slf4j
public class TaskService {

    private final TaskRepository taskRepository;
    private final RabbitTemplate rabbitTemplate;

    @Value("${rabbitmq.exchange.name}")
    private String exchangeName;

    @Value("${rabbitmq.routing.created}")
    private String createdRoutingKey;

    @Transactional
    public Task createTask(Task task) {
        if (task.getTitle() == null || task.getTitle().isEmpty()) {
            throw new IllegalArgumentException("Título da tarefa é obrigatório");
        }
        task.setStatus(TaskStatus.PENDING);
        task.setCreatedAt(LocalDateTime.now());
        task.setUpdatedAt(LocalDateTime.now());

        Task savedTask = taskRepository.save(task);
        log.info("✅ Tarefa criada: ID {} - {}", savedTask.getId(),
savedTask.getTitle());

        TaskCreatedEvent event = new TaskCreatedEvent(
            savedTask.getId(),
            savedTask.getTitle(),
            savedTask.getDescription(),
            savedTask.getAssignee(),
            savedTask.getPriority().name()
        );
        rabbitTemplate.convertAndSend(exchangeName, createdRoutingKey, event);
        log.info("📤 Evento TASK_CREATED publicado para tarefa {}",
savedTask.getId());

        return savedTask;
```

```
    }
}
```

4.7. Controller (TaskController.java)

```
package com.example.task.controller;

import com.example.task.model.Task;
import com.example.task.repository.TaskRepository;
import com.example.task.service.TaskService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/tasks")
@RequiredArgsConstructor
public class TaskController {

    private final TaskService taskService;
    private final TaskRepository taskRepository;

    @PostMapping
    public ResponseEntity<Task> createTask(@RequestBody Task task) {
        Task created = taskService.createTask(task);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @GetMapping
    public ResponseEntity<List<Task>> getAllTasks() {
        return ResponseEntity.ok(taskRepository.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Task> getTask(@PathVariable Long id) {
        return taskRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

4.8. Classe Principal (TaskServiceApplication.java)

```
package com.example.task;

import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TaskServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(TaskServiceApplication.class, args);
    }
}
```

---

5. Implementação do Notification Service (Consumidor 1)

5.1. application.properties

```
spring.application.name=notification-service
server.port=8082

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=admin
spring.rabbitmq.password=admin

spring.datasource.url=jdbc:h2:file:./data/notificationdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update

rabbitmq.exchange.name=task.exchange
rabbitmq.queue.notification=notification.queue
rabbitmq.routing.created=task.created
```

5.2. Modelo Notification.java

```
package com.example.notification.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;

@Entity
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Notification {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private Long taskId;
    private String taskTitle;
    private String recipient;
    private String channel;
    private String message;
    private LocalDateTime sentAt;
    private Boolean success;
}
```

5.3. NotificationRepository.java

```
package com.example.notification.repository;

import com.example.notification.model.Notification;
import org.springframework.data.jpa.repository.JpaRepository;

public interface NotificationRepository extends JpaRepository<Notification, Long>
{
}
```

5.4. RabbitMQConfig (Notification Service)

```
package com.example.notification.config;

import org.springframework.amqp.core.*;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    @Value("${rabbitmq.exchange.name}")
    private String exchangeName;

    @Value("${rabbitmq.queue.notification}")
    private String notificationQueue;

    @Value("${rabbitmq.routing.created}")
    private String createdRoutingKey;

    @Bean
    public TopicExchange taskExchange() {
        return new TopicExchange(exchangeName);
    }

    @Bean
```

```
    public Queue notificationQueue() {
        return new Queue(notificationQueue, true);
    }

    @Bean
    public Binding notificationBinding() {
        return BindingBuilder.bind(notificationQueue())
                .to(taskExchange())
                .with(createdRoutingKey);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

5.5. NotificationService.java

```
package com.example.notification.service;

import com.example.notification.model.Notification;
import com.example.notification.repository.NotificationRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;

@Service
@RequiredArgsConstructor
@Slf4j
public class NotificationService {

    private final NotificationRepository notificationRepository;

    public void sendNotification(Long taskId, String taskTitle, String assignee,
String priority) {
        String channel = priority.equals("CRITICAL") ? "SMS" : "EMAIL";
        String message = String.format("📋 Nova tarefa: '%s' (Prioridade: %s) para
%s",
                taskTitle, priority, assignee);

        log.info("📨 [NOTIFICAÇÃO] Canal: {} | Mensagem: {}", channel, message);

        Notification notif = new Notification();
        notif.setTaskId(taskId);
        notif.setTaskTitle(taskTitle);
        notif.setRecipient(assignee);
        notif.setChannel(channel);
        notif.setMessage(message);
        notif.setSentAt(LocalDateTime.now());
```

```
        notif.setSuccess(true);
        notificationRepository.save(notif);

        log.info("✅ Notificação registrada para tarefa {}", taskId);
    }
}
```

5.6. NotificationEventListener.java

```
package com.example.notification.listener;

import com.example.task.event.TaskCreatedEvent;
import com.example.notification.service.NotificationService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
@Slf4j
public class NotificationEventListener {

    private final NotificationService notificationService;

    @RabbitListener(queues = "${rabbitmq.queue.notification}")
    public void handleTaskCreated(TaskCreatedEvent event) {
        log.info("📥 Notification Service recebeu TASK_CREATED: ID {}",
event.getTaskId());
        notificationService.sendNotification(
            event.getTaskId(),
            event.getTitle(),
            event.getAssignee(),
            event.getPriority()
        );
    }
}
```

5.7. Classe Principal

```
package com.example.notification;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class NotificationServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(NotificationServiceApplication.class, args);
```

```
    }
}
```

---

6. Implementação do Analytics Service (Consumidor 2)

6.1. application.properties

```
spring.application.name=analytics-service
server.port=8083

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=admin
spring.rabbitmq.password=admin

spring.datasource.url=jdbc:h2:file:./data/analyticsdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update

rabbitmq.exchange.name=task.exchange
rabbitmq.queue.analytics=analytics.queue
rabbitmq.routing.created=task.created
```

6.2. Modelo TaskMetric.java

```
package com.example.analytics.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDate;

@Entity
@Data
@AllArgsConstructor
@NoArgsConstructor
public class TaskMetric {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDate date;
    private Long totalTasks;
    private Long highPriorityTasks;
    private Long mediumPriorityTasks;
    private Long lowPriorityTasks;
```

```
    private Long criticalPriorityTasks;
}
```

6.3. TaskMetricRepository.java

```
package com.example.analytics.repository;

import com.example.analytics.model.TaskMetric;
import org.springframework.data.jpa.repository.JpaRepository;
import java.time.LocalDate;
import java.util.Optional;

public interface TaskMetricRepository extends JpaRepository<TaskMetric, Long> {
    Optional<TaskMetric> findByDate(LocalDate date);
}
```

6.4. RabbitMQConfig (Analytics Service)

```
package com.example.analytics.config;

import org.springframework.amqp.core.*;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    @Value("${rabbitmq.exchange.name}")
    private String exchangeName;

    @Value("${rabbitmq.queue.analytics}")
    private String analyticsQueue;

    @Value("${rabbitmq.routing.created}")
    private String createdRoutingKey;

    @Bean
    public TopicExchange taskExchange() {
        return new TopicExchange(exchangeName);
    }

    @Bean
    public Queue analyticsQueue() {
        return new Queue(analyticsQueue, true);
    }
```

```
    @Bean
    public Binding analyticsBinding() {
        return BindingBuilder.bind(analyticsQueue())
                .to(taskExchange())
                .with(createdRoutingKey);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

6.5. AnalyticsService.java

```
package com.example.analytics.service;

import com.example.analytics.model.TaskMetric;
import com.example.analytics.repository.TaskMetricRepository;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDate;

@Service
@RequiredArgsConstructor
@Slf4j
public class AnalyticsService {

    private final TaskMetricRepository metricRepository;

    @Transactional
    public void recordTaskCreated(String priority) {
        LocalDate today = LocalDate.now();
        TaskMetric metric = metricRepository.findByDate(today)
                .orElse(new TaskMetric());

        metric.setDate(today);
        metric.setTotalTasks(metric.getTotalTasks() != null ?
metric.getTotalTasks() + 1 : 1);

        switch (priority) {
            case "CRITICAL" -> metric.setCriticalPriorityTasks(
                metric.getCriticalPriorityTasks() != null ?
metric.getCriticalPriorityTasks() + 1 : 1);
            case "HIGH" -> metric.setHighPriorityTasks(
                metric.getHighPriorityTasks() != null ?
metric.getHighPriorityTasks() + 1 : 1);
            case "MEDIUM" -> metric.setMediumPriorityTasks(
                metric.getMediumPriorityTasks() != null ?
```

```
metric.getMediumPriorityTasks() + 1 : 1);
            case "LOW" -> metric.setLowPriorityTasks(
                metric.getLowPriorityTasks() != null ?
metric.getLowPriorityTasks() + 1 : 1);
        }

        metricRepository.save(metric);
        log.info("📊 Métrica atualizada: {} | Total: {} | Críticas: {}",
            today, metric.getTotalTasks(), metric.getCriticalPriorityTasks());
    }
}
```

6.6. AnalyticsEventListener.java

```
package com.example.analytics.listener;

import com.example.task.event.TaskCreatedEvent;
import com.example.analytics.service.AnalyticsService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
@Slf4j
public class AnalyticsEventListener {

    private final AnalyticsService analyticsService;

    @RabbitListener(queues = "${rabbitmq.queue.analytics}")
    public void handleTaskCreated(TaskCreatedEvent event) {
        log.info("📥 Analytics Service recebeu TASK_CREATED: ID {}",
event.getTaskId());
        analyticsService.recordTaskCreated(event.getPriority());
    }
}
```

6.7. Classe Principal

```
package com.example.analytics;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AnalyticsServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AnalyticsServiceApplication.class, args);
```

```
    }
}
```

---

7. Passo a Passo — Execução e Teste

Passo 7.1 — Iniciar o RabbitMQ (Docker)

```
docker-compose up -d
```

Verifique no navegador: `http://localhost:15672` (login: `admin`/`admin`)

Passo 7.2 — Iniciar os Serviços Spring Boot

Ordem	Serviço	Porta	Comando (na pasta do serviço)
1	`analytics-service`	8083	`mvn spring-boot:run`
2	`notification-service`	8082	`mvn spring-boot:run`
3	`task-service`	8081	`mvn spring-boot:run`

> **Importante:** Os consumidores (analytics e notification) devem ser iniciados antes do produtor (task) para garantir que as filas sejam declaradas.

Passo 7.3 — Criar uma Tarefa via API

Método	URL	Body (JSON)
POST	`http://localhost:8081/api/tasks`	`{"title": "Implementar EDA", "description": "Criar sistema de tarefas com RabbitMQ", "assignee": "João Silva", "priority": "HIGH"}`

Resposta esperada:

```
{
  "id": 1,
  "title": "Implementar EDA",
  "description": "Criar sistema de tarefas com RabbitMQ",
  "assignee": "João Silva",
  "priority": "HIGH",
  "status": "PENDING",
  "createdAt": "2026-07-20T10:00:00",
  "updatedAt": "2026-07-20T10:00:00"
}
```

Passo 7.4 — Verificar os Logs

Task Service:

```
✅ Tarefa criada: ID 1 - Implementar EDA
📤 Evento TASK_CREATED publicado para tarefa 1
```

Notification Service:

```
📥 Notification Service recebeu TASK_CREATED: ID 1
📧 [NOTIFICAÇÃO] Canal: EMAIL | Mensagem: 📋 Nova tarefa: 'Implementar EDA'
(Prioridade: HIGH) para João Silva
✅ Notificação registrada para tarefa 1
```

Analytics Service:

```
📥 Analytics Service recebeu TASK_CREATED: ID 1
📊 Métrica atualizada: 2026-07-20 | Total: 1 | Críticas: 0
```

Passo 7.5 — Verificar no RabbitMQ Manager

Passo	Ação
1	Acesse `http://localhost:15672`
2	Vá para a aba **Queues**
3	Verifique as filas: `notification.queue` e `analytics.queue`
4	Veja as métricas: "Ready" (mensagens aguardando) e "Total" (mensagens processadas)

Passo 7.6 — Consultar Dados nos Serviços

Serviço	Endpoint	Exemplo
Task	`GET /api/tasks`	`http://localhost:8081/api/tasks`
Notification	(adicional) `GET /api/notifications`	`http://localhost:8082/api/notifications`
Analytics	(adicional) `GET /api/metrics`	`http://localhost:8083/api/metrics`

8. Exercícios Complementares (Para os Grupos)

#	Exercício	Descrição	Desafio
1	**Adicionar Filas com Routing**	Modifique para que tarefas críticas tenham uma fila separada	Use **Topic Exchange** com padrões (ex: `task.created.critical`)
2	**Adicionar Serviço de Log**	Crie um 3º consumidor que registra todos os eventos em um arquivo	Use uma Fanout Exchange para broadcast
3	**Reprocessamento**	Simule falha no Notification Service e reinicie — mensagens devem ser reentregues	Configure `acknowledge-mode="manual"`
4	**Idempotência**	Garanta que o Analytics Service não duplique métricas	Use `eventId` para deduplicação
5	**Dead Letter Queue**	Configure uma DLQ para mensagens que falharem após múltiplas tentativas	Configure `x-dead-letter-exchange`
6	**Tarefa Atualizada**	Implemente `TASK_UPDATED` e notifique sobre atualizações	Adicione endpoint PUT para atualizar tarefas
7	**Dashboard**	Crie endpoint no Analytics Service que retorna resumo das métricas do dia	Use `Projection` ou DTO customizado

9. Solução de Problemas Comuns

Problema	Possível causa	Solução
`Connection refused`	RabbitMQ não está em execução	Execute `docker-compose up -d` e verifique com `docker-compose ps`
Mensagem não é consumida	Fila não está vinculada corretamente	Verifique se o binding usa a routing key correta
`ClassNotFoundException` no Consumer	Pacote do evento não é o mesmo	Copie a classe `TaskCreatedEvent` para todos os serviços
`AmqpRejectAndDontRequeueException`	Erro no processamento do evento	Verifique a lógica do método `handleTaskCreated` e os logs
H2 console não aparece	Dependência ausente	Verifique se `h2` está no `pom.xml`
Porta em uso	Outro serviço rodando na mesma porta	Altere `server.port` no `application.properties`

10. Verificação Final (Checklist)

Item	Descrição	Feito?
1	`docker-compose.yml` criado e RabbitMQ em execução	☐
2	`task-service` configurado e rodando na porta 8081	☐
3	`notification-service` configurado e rodando na porta 8082	☐
4	`analytics-service` configurado e rodando na porta 8083	☐
5	Tarefa criada via POST com sucesso	☐
6	Notification Service recebeu o evento e registrou notificação	☐
7	Analytics Service recebeu o evento e atualizou métricas	☐
8	Filas `notification.queue` e `analytics.queue` visíveis no RabbitMQ Manager	☐
9	Logs de todos os serviços mostram o fluxo completo	☐
10	Consulta GET retorna os dados persistidos	☐

Let's Go!